

Instructor: Date:

Learner: Regional Branch:

Program: Center:

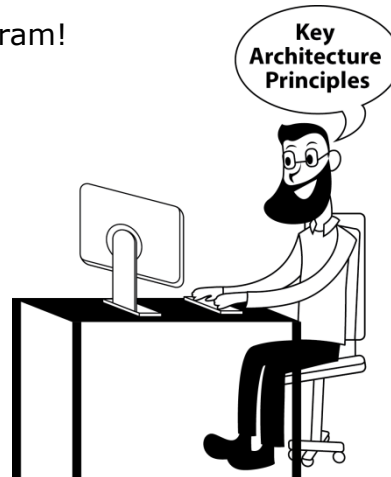
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

I don't understand this diagram!



Learning Outcome:

Interpret the technical design report, to determine the work plan during the phase of building the software, based on regulation and protocols established by the company.

Design the architecture design using the appropriate tools and types according to the software requirements.

Can Do Statements:

- I can understand written information about the key architecture and design principles.
- I can answer questions related with key architecture and design principles.
- I can write a short paragraph about key architecture and design principles.

Ask your partner:

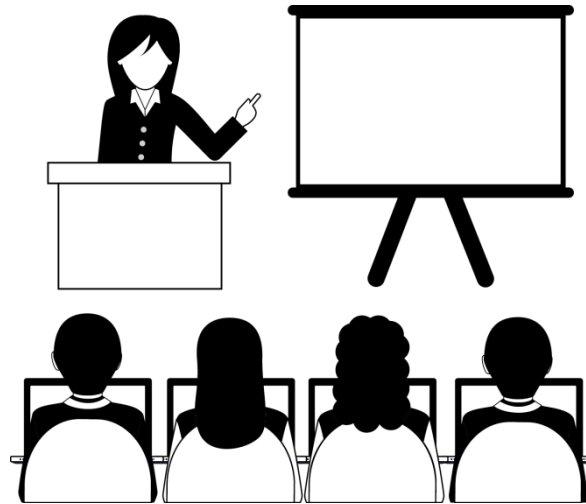
- Is it important to be organized when creating software architectures? Why?
- How can you make the others understand the codes and structures you have used in your architecture?





Let's Explore!

Jennifer is reading an article about software optimization, what is the main idea?



Instructions: Read the text below.

Now you've decided to build software and specialized apps for your business, you have got to know that there are some basic and important principles that you will need to bear in mind to succeed in the process of creating a good and stable architecture. Such principles are going to ease your life and the process of constructing the architecture, but when we say principles, we have got to talk about two main groups, architecture and design principles. The first one is related to the architecture itself and the second one will help you minimize costs and requirements while maximizing the architecture.

We can say that among the many ways of maximizing performance, we have chosen some that will help you boost your architecture, like, firstly, take into account when you are building an architecture you will need to build it to change instead of last, that means you are going to have to build the architecture to suffer changes, so that it addresses new requirements. That, on the other hand, means you are going to have to use an iterative approach to add details and ensure the architecture will be perfectly optimized.

One of the main question that arise when talking about architectural design is how you are going to maximize the usability and extendibility of the architecture. Well, that is not a cakewalk, but not unbearable either. Firstly, you are going to have to separate the concerns, that is, to divide components into specific features. You may want to group the components into logical layers to understand the structures of the system in an organized way, another way to maximize your architecture is by restricting knowledge with the principle of least knowledge, this way, you avoid objects having key information of other components inside them.





Let's Practice!

The key principles allow you to improve your designs.

Instructions: Match column A with Column B.

COLUMN A

1. Model to analyze.
2. Single responsibilities.
3. Minimize large design.
4. Never repeat functionalities.
5. Define communication protocols.

COLUMN B

- a. You have got to use UML to design decisions and analyze its impacts.
- b. You haven't got to make large designs that make it difficult to change them in the future.
- c. You have got to set how the layers and components interact and communicate.
- d. You are going to have to use each code with a unique line that avoids its functionality from duplicating.
- e. You are going to have to use each module to understand the program with a specific responsibility.



Let's Practice!

Let's know the key principles of software architecture!

Instructions: Match the sentences from Column A with Column B.

COLUMN A

1. How are you going to have to build the architecture of a software?
2. When have you got to add details and ensure the optimization of the architecture?
3. Why have you got to separate the concerns? Because
4. Why are you going to have to use key design principles?
5. What key architectural principles have you got to use?

COLUMN B

- a. I am going to build the architecture of the software in order to avoid changes through time.
- b. The principles are going to ease our life and the process of constructing the architecture.
- c. I have got to use the principles that relate to the structure and the technical and operational requirements.
- d. When I have got to use an iterative approach.
- e. I have got to divide the components according to their features.



Extension Activity

Let's know more about the principles!

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Research for all the key architectural principles.
- Look for a web application.
- Describe its architecture.
- List all the characteristics.
- Write a short paragraph describing all the principles you found in the web application.
- Share it with a partner.

Self-Assessment

Yes

No

- I do research on specific topics.
- I describe a web application's architecture.
- I list characteristics.
- I write a short paragraph explaining all the principles a web application may have.
- I discuss my work with a partner.

☐ ☐
☐ ☐
☐ ☐
☐ ☐


Target Words

- **succeed** (verb): to triumph over something or someone
e.g. There are some basic and important principles that you will need to bear in mind to succeed in the process of creating a good and stable architecture.
- **iterative** (adjective): done in a repetitive way
e.g. You are going to have to use an iterative approach.
- **extendibility** (noun): the process of being enlarged
e.g. You are going to maximize the usability and extendibility of the architecture.
- **cakewalk** (idiom): to be really easy
e.g. That is not a cakewalk.
- **unbearable** (adjective): to be impossible to happen
e.g. Well, that is not a cakewalk, but not unbearable either.
- **restrict** (verb): to limit or avoid to happen
e.g. Another way to maximize your architecture is by restricting knowledge.



Learning Points

Have Got to and Going to Have to for What Is Necessary

You can use **have got to** or **be going to have to** when expressing necessity either in present or future ideas.

You can use **have to** or **have got to** in order to show necessity in present ideas, however have got to is more informal and used in spoken English.

Example:

*You **have (got) to** have an efficient communication when transmitting the design to others.*

In interrogative sentences you use **have to** with the auxiliary **do/does**, but with **have got to**, you place the auxiliary **have/has** at the beginning of the sentence.

Example:

Have you got to** change the architecture? – No, **I don't have to!

Do you have to** change the architecture? – No, **I don't have to!

Notice you use **have to** rather than **have got to** in negative statements.

You can use the auxiliary be going to or will to express the idea of necessity in the future.

Example:

*You **are not going to have to** make last minute changes that can affect the design if you follow the instructions.*

Examples

- *You have got to know that there are some basic and important principles.*
- *We have got to talk about two main groups.*
- *You are going to have to build the architecture to suffer changes.*
- *You are going to have to use an iterative approach.*





Learning Points

Key Architectural and Design Principles

You use **architectural and design principles** to maximize the durability of the architecture and minimize costs among others.

There are some key principles you might want to consider when you come to create software architecture.

See chart A

There are also some other design aspects you might want to analyze so that you can maximize the architectural performance while minimizing costs and maintenance requirements.

See chart B

Key architectural principles	
Build to change	Create your apps to change and adapt to future changes.
Model to Analyze	You can use UML modeling systems to design decisions and analyze the positive and negative impacts.
Use Communication and Collaboration Tools	All possible models you use can work as a communication and collaboration system among the stakeholders.
Iterative Approach	It is always good to start with a baseline architecture so that you can evolve it by repetitive testing to ensure perfection.

Chart B





Learning Points

Key Design Principles	
Separate components	Divide the components into individual objects with their own responsibilities to avoid overlapping among them.
Single responsibilities	Give each module a specific responsibility that helps the user understand the program.
The least knowledge the better	Components should never have key or important information about other components within them.
Minimize large designs	Avoid large designs, so that if there are changes to be done, the modifying process will be easier.
Never repeat functionalities	Components can't be repeated as each code has a unique code line that prevents duplication of functionality.
Group components into layers	Logical layers help the user to understand the organization of the design and how the components are distributed inside the architecture.
Define communication protocols	It is important to set how layers and components are going to communicate to each other.
Set Data formats	It is really important to have same data formats inside layers because there are many components that will interact with those formats, if you keep the same data formats, the application will be easy to maintain and extend.
Manage Exceptions	When you define exceptions, you avoid errors related to security from happening.
Conventions	Naming conventions are really handy for all team members and users to understand the system easily.

